CS34020/CSM4220 Workshop 1

Foundations of Vision

Theme: Space Science

Purpose and theme: in this workshop we will spend some time getting to know OpenCV, a popular open-source library for Computer Vision - https://opencv.org. The tasks in this workshop are all around the theme of *space science* but could be applied in other domains.

Part 1: Meet Spyder and OpenCV

For these workshops we are going to be using the most popular open-source library for Computer Vision: **OpenCV**. We're going to be working in Python as this is easy to get started with and is quite an extendable ecosystem, offering us packages for plotting graphs and more, which will be very helpful. There are also lots of tutorials online for Python and OpenCV, so it be a good place to start if you want to explore more.

> Your most important reference is the OpenCV Python docs:
> https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html

Have a look through these to start with; we will refer back to particular parts later, but again it's a useful resource generally if you want to learn more!

For these workshops I recommend using **Spyder**, it's installed on the machines in the lab and is configured ready for use. If you want to use your own laptop and/or a different IDE then you can, but you will need to work out how to port these instructions to your setup.

Open Spyder and have a look at the interface – it's fairly intuitive, but there are lots of tabs and panels that it is worth looking through. Let's start by running a short bit of code to check that OpenCV is working correctly. Type the following into a file, then run it by clicking the green 'play' arrow in the top menu bar.

```
import cv2 print(cv2.__version__)
```

If all goes well, the IPython Console in the bottom right will give a message like this:

```
runfile('/...path.../testOpenCV.py', wdir='/...path.../')
4.6.0
```

This is telling us that it's running this particular file from the noted working directory, and that the version of OpenCV I have is 4.6.0 – all is working!

Part 2: Open up a file (safely)[1]

Download the sample image files from Blackboard and put them into the same place you will save your Python files.

Create a new python file – we are going to start by just loading in an image and displaying it.

Step 1. Import the OpenCV library into your Python file.
Step 2. Read the image into a variable so that you can work with it.
Step 3. Show the image in a Python window, and clean everything up afterwards. This is the outline of the code to follow these steps:

```
import cv2                                  # step 1
img = cv2.imread("yourfile.jpeg")           # step 2
# step 3
cv2.imshow("img", img)    # args = window title, file to show
cv2.waitKey(0)            # pauses the program
cv2.destroyAllWindows()   # clean up at the end
```

Pick one of the sample files and have a go at implementing this. If the file is in the same folder as your Python script then you can just use the file name and extension. If you want to store the image in a different directory then you can give `imread` a relative or absolute path instead, for example:

```
img = cv2.imread("../path/to/yourfile.jpeg")
```

You can also choose to explicitly import the image in greyscale by passing the `cv2.IMREAD_GRAYSCALE` argument after the file name, e.g.:

```
img = cv2.imread("yourfile.jpeg", cv2.IMREAD_GRAYSCALE)
```

You might find that if you got the image path wrong that Python gave an error and crashed. It would be a good idea for us to set up a tidy way to deal with this before going further. One simple way to do this is to use an assert to check the `img` value:

```
assert img is not None, "file could not be read"
```

Alternatively, you can use the sys package (just the highlighted lines):

```
import cv2                              # step 1
import sys
img = cv2.imread("yourfile.jpeg")  # step 2
if img is None:
    sys.exit("Could not read the image.")
```

If Python can't load in a file then the `img` variable will be empty, and it will exit the program while giving you a useful error message.

[1] This OpenCV Tutorial that covers similar topics, you might want to look through for extra content: https://docs.opencv.org/4.x/d3/df2/tutorial_py_basic_ops.html

Images have lots of useful data associated with them, try the following:

```
# Getting some image properties
imginfo = "Image properties:\n" + \
          "Shape = " + str(img.shape) + \
          "\nNumber of pixels = " + str(img.size) + \
          "\nImage datatype = " + str(img.dtype)
print(imginfo)

# output the raw image data to console
print(img)
print(img[0][0][0])
```

- **shape** will give you the size of the image in rows, columns, and colour channels; note that if you import a greyscale image then the channels will be 1 (and asking for them may throw an error).
- **size** gives you the number of pixels in your image.
- **dtype** will output the datatype of the image file, e.g. uint8.

You can print the pixel values to the console, in particular it's useful to know you can access individual pixels and channel data in the format `img[row][col][channel]`.

Part 3: Some Basic Image Processing

There is a set of four images of the sun on Blackboard:

- Taking_the_Sun_s_temperature_630k.png
- Taking_the_Sun_s_temperature_320k.png
- Taking_the_Sun_s_temperature_32k.png
- Taking_the_Sun_s_temperature_10k.png

Read in the four images each into their own variable, we are going to experiment with doing **arithmetic** on them.

First: adding images together. There are multiple ways of doing this using OpenCV, try each of the following with your images and think about what the differences are:

```
result = img1 + img2
result = cv2.add(img1,img2)
result = cv2.addWeighted(img1,0.3,img2,0.7,0)
```

Similar functions are available for subtraction, again try these:

```
result = img4 - img3

result = cv2.subtract(img4,img3)
```

Test different combinations to see what kinds of results you get.

You can also work with the images numerically, for example you can increase the brightness of the image by adding a constant:

```
result = img1 + 25
```

We might only want to use portions of an image, some useful ways of choosing what data we work with are to use **Regions of Interest (ROIs)**, **thresholding**, and **masking**. An ROI is a smaller piece of the whole image which you want to focus on. Because images are just a matrix of pixel values, we can treat them like any other matrix (or array) and ask for a subset. We can use the values as variables, for example:

```
rows,cols,channels = img.shape
halfheight = round(rows/2)
img1 = img[0:halfheight, 0:cols]
```

We also often need to resize an image or an ROI, this can be done using the `resize` function:

```
resized_image = cv2.resize(image, (new_width, new_height))
```

When using deep neural networks (which we will look at later), the input often needs to be of the correct dimensions. It is also important to *normalise* the images in the same way as was done during training, for example to include normalising the brightness and contrast to standard values. The input to deep neural networks generally also have an additional dimension for the "batch" of images, and the order of dimensions is often different to the opencv image arrangement of [height, width, channels] e.g. [batch, channels, height, width]. OpenCV provides a function `blobFromImage` in the **dnn** package, which performs resizing, normalisation, and reordering. We can see the effect of normalising using this function by creating a "blob" (i.e. normalised image) from one of our example images, and reordering the packing of the image (using the transpose function) i.e.

```
# Set up some variables for the target shape, mean and scale
input_shape = (300, 300)
mean = (127.5, 127.5, 127.5)
scale = 0.007843
# use the blobFromImage method to do the normalisation
blob = cv.dnn.blobFromImage(img1, scalefactor=scale, \
                size=input_shape, mean=mean, swapRB=True)
blob = blob.transpose([0,2,3,1])[0] # change the blob's shape
cv.imshow("Blob", blob) # Show the blob as an image
```

The next snippet of code shows an example of both masking and thresholding[2]:

[2] These OpenCV Tutorials that covers similar topics, you might want to look through for extra content:

```
# import the mask like a normal image
imgmask = cv2.imread("yourmaskfile.png", cv2.IMREAD_GRAYSCALE)
# ensure it's binary using a thresholding operation
ret,imgmask = cv2.threshold(imgmask,245,255,cv2.THRESH_BINARY)

result = cv2.bitwise_and(img, img, mask = imgmask)
```

You start by importing your mask just like any other image. In this example we use a binary threshold on the mask image to make sure it is a binary image. You can specify the type of thresholding to be done, along with the cut-off values to use. Have a look at the following for a detailed explanation of the thresholding types: https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html In the module we have only looked at basic thresholding, but this tutorial also shows some more advanced thresholding options that are worth trying. Two masking files have been provided to go with the four solar images;

- Taking_the_Sun_s_temperature_mask.png
- Taking_the_Sun_s_temperature_mask2.png

Try applying the masks to the images. One of the masks covers the numbers in the corner of the image, while the other covers the centre of the sun and leaves only the outer edge (the corona) visible. Experiment with masking one of the images and then applying some of the other techniques you have tried so far to see if you can make the corona more visible and highlight the details.

Part 4: Playing with colour

The first thing to note about colour in OpenCV is that it uses BGR as the standard order for colour channels (rather than RGB). You can split and merge colour channels like this:

```
b,g,r = cv2.split(img)

result = cv2.merge((b,g,r))
```

Try this snippet, but change the order of the channels in the merge to see what happens.

You can merge channels from different images (if they are the same size). Try loading in the following three images in greyscale then assigning them to be following colour channels:

- Red = Taking_the_Sun_s_temperature_630k.png
- Green = Taking_the_Sun_s_temperature_320k.png
- Blue = Taking_the_Sun_s_temperature_32k.png

You can create false-colour images in this way, and indeed this is roughly how many space images are given colour (e.g. galaxy images) because some detail may not be visible to the human eye.

You can convert images to different **colour spaces**, for example HSV or L*a*b* using the `cvtColor` function, here are a couple of examples:

---

https://docs.opencv.org/4.x/d0/d86/tutorial_py_image_arithmeIcs.html
https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html

```
imghsv = cv2.cvtColor(result, cv2.COLOR_BGR2HSV)

imglab = cv2.cvtColor(result, cv2.COLOR_BGR2LAB)
```

There are a very large number of options for which spaces you can convert[3] between, you can see a list of them here: https://docs.opencv.org/4.x/d8/d01/group__imgproc__color__conversions.html#ga4e0972be5de079fed4e3a10e24ef5ef0.

If you convert to a non-RGB colour space then display it with the `imshow` function, the image will look a little different – the data will be correctly converted, but imshow assumes the image is in BGR format so it tries to display the data in that format even though it no longer makes sense.

Colour spaces become especially interesting if you split the result and look at the single channels. Try looking at the L*a*b* and HSV channels, you should be able to see why these colour spaces are very useful in Computer Vision, particularly the Lightness channel in L*a*b* and the Value channel in HSV.

Link to the Assignment

In the assignment you are asked to detect yellow cars. Consider the following questions relating to the assignment, and **write down your answers**, you will need them for the assignment report.

Questions for CS34020 Students

What Python / opencv functionality from this workshop could be used to:

- load an image from file and display it?
- prepare an image for deep learning car detection?
- crop a rectangle from the image?
- change the colour space of an image?

Questions for CSM4220 Students

Find a recent academic paper (last 5 years) that makes use of some (at least 2) of the algorithms used in this workshop i.e. image arithmetic, image region of interest extraction, image normalisation (such as size, brightness, contrast), image thresholding, image colour space conversion. Note, the algorithms don't need to be the main focus of the paper, but it should be clear that they make use of the algorithms. Explain briefly in 1 short paragraph how the algorithms are used in the paper and cite the paper.

Submission

Please **submit your work from this workshop**, to Blackboard at the end of the practical. The submissions themselves are not assessed and carry no marks, but will be used as an additional measure of **engagement**. Don't worry if you haven't completed all aspects of this worksheet, just submit what you have done.

[3] There is a lot more detail on the formulas that allow these conversions here: https://docs.opencv.org/4.x/de/d25/imgproc_color_conversions.html